

Del Pos Especializado al ERP Modular: Hoja de Ruta para Escalar Bocatta POS a una Plataforma Multi-Giro

Este informe presenta un análisis exhaustivo del estado actual del proyecto Bocatta POS, documentando su evolución desde una aplicación especializada en alimentos hasta una plataforma modular preparada para escalar a múltiples giros comerciales. Se detallan los fundamentos arquitectónicos, los mecanismos para gestionar la complejidad operativa, la estrategia de persistencia de datos y sincronización offline, así como el estado de la calidad del código, la infraestructura y el sistema de diseño. Finalmente, se identifican las brechas existentes y se propone una hoja de ruta clara para alcanzar un nivel de madurez superior.

Fundamentos Arquitectónicos: La Transición hacia un Modular Monolith

La transformación más significativa en el desarrollo de Bocatta POS ha sido la adopción de una arquitectura Modular Monolith, una estrategia que busca combinar la simplicidad de un monolito con la flexibilidad y escalabilidad de un sistema distribuido. Esta aproximación permite organizar el código en módulos cohesivos, cada uno con un contrato de dependencia claro, lo que facilita el mantenimiento, la prueba y el desarrollo independiente de características complejas. El núcleo de esta arquitectura reside en una capa de dominio pura, completamente libre de dependencias externas a Android o Firestore, lo que constituye la piedra angular de la escalabilidad y la estabilidad del sistema.

La base de esta arquitectura se construyó en la capa de dominio (`domain/`), donde se definieron interfaces para encapsular la lógica de negocio. Se crearon contratos como `IProductRepository` y `IInventoryRepository`, que especifican qué operaciones pueden realizarse sobre los productos e inventario sin definir cómo se llevan a cabo. Esto permite que la capa de presentación (UI) y otros componentes del dominio interactúen con estos contratos, mientras que la capa de datos (`data/`)

proporciona las implementaciones concretas que se conectan a la base de datos. Por ejemplo, `ProductMappers.kt` contiene funciones para convertir entre el modelo legacy (`ProductoV2`) y el nuevo modelo de dominio (`ProductV2`), manteniendo la integridad de los datos durante la transición sin que el dominio sea consciente de la existencia del modelo antiguo `[[unique_id]]`. Esta separación estricta es fundamental, ya que cualquier cambio en la implementación de persistencia (por ejemplo, pasar de Firebase a una base de datos local) solo afectaría a la capa de datos, dejando intacta la lógica de negocio del dominio.

Un desafío técnico crucial surgió durante este proceso: la incompatibilidad entre la versión de Kotlin 2.2.10 utilizada y las últimas versiones de KSP (el procesador de anotaciones de Kotlin para Room). Intentar usar Room para la persistencia offline resultaba en un error de compilación debido a conflictos con el soporte integrado de Kotlin en Android Gradle Plugin 8.x `[[unique_id]]`. En lugar de revertir a una versión anterior de Kotlin o abandonar la persistencia offline, se tomó una decisión pragmática y robusta: migrar la cola de ajustes de inventario a una implementación nativa de SQLite utilizando `SQLiteOpenHelper`. Esta solución, aunque requiere más código manual, eliminó por completo la dependencia de herramientas externas, permitiendo que el proyecto se compilara sin problemas y preservando la arquitectura limpia. La interfaz de dominio `IStockAdjustmentQueue` fue clave aquí; tanto la implementación de Room como la nueva de SQLite nativa la siguen, lo que significa que el resto del sistema no tuvo que cambiar ni saber cuál de las dos estaba siendo utilizada `[[unique_id]]`. Esta resolución demuestra un alto nivel de madurez técnica, priorizando la estabilidad del sistema sobre la conveniencia de una herramienta específica.

La siguiente tabla resume la estructura de capas arquitectónica y sus responsabilidades:

Capa	Ubicación Principal	Responsabilidades Clave
Presentación (UI)	presentation/	Manejo de estado de la UI, renderizado de componentes Compose, llamadas a casos de uso, gestión de eventos del usuario. No contiene lógica de negocio <code>[[unique_id]]</code> .
Capa de Dominio	domain/	Definición de entidades, interfaces de repositorio (<code>IRepository</code>), casos de uso (<code>UseCase</code>), motores de lógica pura (<code>Engine</code>). Contiene la regla de negocio central <code>[[unique_id]]</code> .
Capa de Datos	data/	Implementación concreta de las interfaces de dominio. Interacción con fuentes de datos (Firebase, SQLite). Mapeo de modelos de datos a modelos de dominio <code>[[unique_id]]</code> .

Esta estructura modular ha demostrado ser efectiva para gestionar la complejidad. Por ejemplo, el motor de producción (`ProductionEngine`) puede tener implementaciones específicas para diferentes giros comerciales (`FoodProductionEngine`, `RetailKitEngine`, `ServiceTaskEngine`) `[[unique_id]]`. La `BusinessLogicFactory` se encarga de devolver la implementación correcta según la

configuración del giro del negocio, manteniendo la UI agnóstica a estas implementaciones concretas `[[unique_id]]`. Al seguir los principios de Clean Architecture, Bocatta POS ha creado una base sólida que no solo es fácil de mantener, sino también preparada para adaptarse a nuevos mercados y requerimientos futuros sin necesidad de refactorizaciones masivas.

Gestión de la Complejidad Operativa: Precisión de Datos y Separación de Negocios

Para que un sistema POS/EPR sea fiable, especialmente en un entorno comercial real, debe manejar con precisión matemática las transacciones financieras y distinguir claramente entre la lógica financiera y la física. Bocatta POS ha abordado estos dos aspectos críticos de manera robusta, sentando las bases para una plataforma precisa y confiable. La estrategia se centra en dos pilares: la separación estricta entre la lógica de descuentos (financiera) y la lógica de deducción de stock (física), y el uso de `BigDecimal` para garantizar la exactitud en todos los cálculos monetarios.

La separación entre lógica financiera y física es quizás el hallazgo más importante para la solidez del sistema. Los descuentos, promociones y la lógica de precios se gestionan exclusivamente a través del `PromotionsEngineV2`. Este motor recibe el carrito de compra y devuelve el total del descuento aplicable, pero nunca toca el inventario `[[unique_id]]`. Por otro lado, el flujo de venta que afecta al stock es manejado por `SalesFlowUseCase`, que delega en `ProductionEngine` y `InventoryRepositoryImpl`. Estos componentes se centran únicamente en cantidades físicas: validan si hay suficiente stock, convierten unidades a una base común usando `UnitConverter`, y ejecutan las transacciones atómicas de descuento en `Firestore` `[[unique_id]]`. Esta separación es vital porque un producto puede ser gratuito por una promoción (descuento total) pero sigue consumiendo material físico (ingrediente de una receta, pieza de un kit). Confundir estas dos dimensiones llevaría inevitablemente a inconsistencias, como registrar una venta sin reducir el inventario correspondiente. La implementación actual respeta esta regla de oro, lo que reduce drásticamente el riesgo de errores de inventario y finanzas.

El segundo pilar es la precisión fiscal. Para eliminar los errores de redondeo inherentes a los números de punto flotante binario (`Double`), el sistema utiliza `BigDecimal` para todas las operaciones monetarias. El `TaxCalculator` y el `FiscalMath` utilitario (`Double.fiscalRound()`) garantizan que los cálculos de impuestos y precios sean

exactos, utilizando `RoundingMode.HALF_UP` que es el estándar para la contabilidad `[[unique_id]]`. Más importante aún, se ha implementado una verificación rigurosa para asegurar la consistencia final: el total del impuesto calculado para una transacción debe ser idéntico a la suma de los impuestos de cada ítem individual (`taxTotal == sum(itemTaxes)`). Para esto, se usa una comparación estricta con `compareTo(BigDecimal.ZERO) == 0` en lugar de la igualdad simple (`==`), que es propensa a errores de precisión incluso con `BigDecimal` si no se gestiona correctamente `[[unique_id]]`. Esta atención al detalle no es un lujo, sino una obligación para cualquier sistema que maneje dinero, ya que previene desviaciones de centavos que pueden acumularse y generar discrepancias en los cierres de caja.

Finalmente, la gestión de unidades de medida es un componente clave para la escalabilidad multi-giro. El `UnitConverter` es un motor puro de dominio que realiza conversiones métricas (g, kg, ml, L, pza) basándose en factores de conversión a una unidad base `[[unique_id]]`. La estrategia adoptada es almacenar todas las cantidades de inventario en esta unidad base dentro de la base de datos. Las conversiones se realizan en la frontera del sistema, cuando se ingresa una cantidad (ej. comprar 2kg de harina) o se muestra una cantidad (ej. mostrar el stock restante en gramos) `[[unique_id]]`. Esta normalización estricta asegura que no haya ambigüedad en el inventario. Un desarrollador que añada un nuevo giro comercial (ej. talleres mecánicos que miden en horas y litros de aceite) puede hacerlo simplemente definiendo las nuevas unidades y sus factores de conversión en el catálogo de unidades, sin alterar la lógica de negocio subyacente. Esta combinación de separación de lógicas, precisión fiscal y gestión de unidades robusta permite a Bocatta POS manejar la complejidad operativa de forma controlada y escalable, preparándolo para operar en un entorno de producción real con alta fiabilidad.

Persistencia de Datos y Sincronización Offline Atómica

Una de las capacidades más críticas para un POS en un entorno comercial real es la capacidad de operar de forma fiable sin conexión a internet. Bocatta POS ha implementado una solución de persistencia offline avanzada que va más allá de simples cachés y se enfoca en garantizar la atomicidad y la integridad de las transacciones, incluso en condiciones de red intermitente. La solución elegida, aunque diferente a la originalmente planeada, es robusta y arquitectónicamente sólida.

El corazón del sistema offline reside en una cola de ajustes de inventario. Cuando una venta ocurre sin conexión, en lugar de intentar una operación compleja en tiempo real, el sistema simplemente encola los detalles del ajuste (qué producto, cuánto, por qué motivo) en una base de datos local. Inicialmente, esta capa de persistencia se buscaba implementar con Room, pero debido a la incompatibilidad de KSP con la versión de Kotlin, se optó por una solución nativa de SQLite utilizando `SQLiteOpenHelper` `[[unique_id]]`. Aunque esto requirió escribir más código manual para la gestión de la base de datos, la ventaja fue inmensa: eliminó por completo la dependencia de herramientas externas que bloqueaban la compilación y permitió continuar con el desarrollo sin comprometer la arquitectura. La clave fue mantener la interfaz de dominio `IStockAdjustmentQueue`, que define los métodos `enqueue`, `getAllPending` y `deleteProcessed`. La implementación de Room y la nueva de SQLite nativo ambas satisfacen esta interfaz, manteniendo la inversión de dependencias y la limpieza del código `[[unique_id]]`.

La característica más sofisticada de esta implementación es el mecanismo de sincronización atómica. Para evitar perder ventas o crear duplicados en caso de fallos, se implementó un patrón de "commit de dos fases". El `SyncWorker`, responsable de la sincronización en segundo plano, opera de la siguiente manera: 1. **Carga:** Lee todos los ajustes pendientes del estado PENDIENTE. 2. **Bloqueo:** Antes de cualquier operación remota, marca todos los ajustes seleccionados en la base de datos local como `IN_PROGRESS`. Esto actúa como un semáforo, indicando que están siendo procesados. 3. **Sincronización:** Intenta enviar cada ajuste a Firestore dentro de una única transacción. Si la transacción tiene éxito, se procede al paso 4. Si falla, se salta al paso 5. 4. **Confirmación:** Después de una sincronización exitosa, se marcan los ajustes como `COMPLETED`, generalmente borrándolos de la cola local. 5. **Reintento:** Si la sincronización falla, se restaura el estado de los ajustes a PENDIENTE en la base de datos local, de modo que puedan ser procesados en el próximo ciclo de sincronización. Todo este proceso se envuelve en un bloque `try...finally` para garantizar que el estado de los ajustes se restablezca correctamente, independientemente de si hubo un fallo o un éxito `[[unique_id]]`.

Este patrón es extremadamente robusto. Evita la situación en la que se borran los registros locales antes de que la escritura remota haya tenido éxito, lo que podría llevar a la pérdida permanente de una venta. También previene la duplicación de movimientos en Firestore, ya que solo se borran los registros de la cola local después de confirmar su éxito en el servidor. Para gestionar la concurrencia y evitar condiciones de carrera en la base de datos SQLite, se utilizó un dispatcher de coroutines limitado a un solo hilo (`Dispatchers.IO.limitedParallelism(1)`), lo que garantiza que las operaciones de la cola se procesen en orden estricto `[[unique_id]]`.

Además, la política de reintentos de WorkManager fue optimizada. En lugar de la retroalimentación lineal predeterminada, se configuró una retroalimentación exponencial con un intervalo base de 5 minutos y un número máximo de reintentos limitado. Esto previene picos de tráfico y consumo de batería en dispositivos POS, haciendo que el sistema sea más eficiente y respetuoso con el hardware `[[unique_id]]`. La combinación de una cola SQLite nativa, un patrón de commit de dos fases en el `SyncWorker` y una política de reintentos inteligente ha creado un subsistema offline que es funcional, fiable y preparado para el despliegue en un entorno de producción real, llenando el vacío dejado por la imposibilidad de usar Room en este momento.

Calidad del Código, Infraestructura y Preparación para la Producción

La calidad del código y una infraestructura de entrega automatizada son tan importantes como la arquitectura para el éxito a largo plazo de un proyecto. Bocatta POS ha realizado progresos significativos en esta área, estableciendo una base sólida para la producción mediante la implementación de un pipeline CI/CD, la adopción de buenas prácticas de linting y un enfoque disciplinado en la gestión de la deuda técnica. El sistema ha alcanzado un estado de madurez donde está listo para ser probado con datos reales, con una calificación de calidad de 97/100.

Un hito fundamental fue la creación de un pipeline de GitHub Actions (`ci.yml`) que automatiza el proceso de integración continua `[[unique_id]]`. Este pipeline ejecuta una serie de pasos críticos de forma sistemática cada vez que se solicita un cambio: verifica el checkout del código, configura el JDK necesario, aplica `ktlint` para mantener la coherencia del estilo de código, ejecuta `detekt` para detectar problemas de calidad y `smell tecnico`, y finalmente, compila el proyecto y ejecuta la suite de pruebas unitarias. El hecho de que el pipeline sea exitoso (187 tests unitarios pasando) y genere un artefacto APK indica que el sistema cumple con un umbral mínimo de calidad antes de ser considerado para la fusión en la rama principal `[[unique_id]]`. Esta automatización elimina la dependencia de revisiones manuales para tareas repetitivas y proporciona una retroalimentación rápida al equipo de desarrollo.

La gestión de la deuda técnica ha sido proactiva y transparente. Cuando surgió el problema de la incompatibilidad de Room/KSP, en lugar de ignorarlo, se documentó cuidadosamente en un archivo `TECH_DEBT.md` `[[unique_id]]`. Este documento no solo describe el problema (Room/KSP no compatible con Kotlin 2.2.10), sino que también

explica el impacto (bloqueo de la compilación de Room), la solución temporal (usar SQLite nativo) y los criterios para resolverlo en el futuro (cuando KSP publique una versión compatible). Esta transparencia es crucial para el plan de desarrollo a largo plazo, ya que permite al equipo tomar decisiones informadas sobre la priorización de tareas. Además, se han implementado scripts de migración para asegurar que los datos existentes se transfieran correctamente entre esquemas, por ejemplo, al pasar de archivos JSON a la base de datos SQLite `[[unique_id]]`.

En cuanto a la calidad del código, se ha logrado una cobertura de pruebas unitarias muy alta para la capa de dominio, con 187 tests que cubren la lógica de negocio pura `[[unique_id]]`. Esto incluye pruebas para el motor de conversión de unidades, el motor de promociones, el caso de uso de flujo de ventas y la cola de ajustes de stock. Sin embargo, la cobertura completa aún no cubre las capas de presentación (UI) y de datos (interacción con Firestore). Falta implementar pruebas instrumentadas que usen el Firestore Emulator Suite para validar la lógica de escritura y lectura bajo distintas condiciones de red, y pruebas de UI con Compose Test para verificar el comportamiento visual y los flujos de usuario `[[unique_id]]`. Estas pruebas son el siguiente paso para cerrar la brecha y obtener una confianza total en todo el sistema.

A continuación se presenta una tabla con las métricas de calidad clave:

Métrica	Estado Actual	Descripción
Pipeline CI/CD	Activo	GitHub Actions con ktlint, detekt, testDebugUnitTest y assembleDebug <code>[[unique_id]]</code> .
Tests Unitarios	187 tests pasando	Cobertura alta en la capa de dominio, pero faltan pruebas de UI e integración <code>[[unique_id]]</code> .
Limpieza de Código	Limpieza	Configurado y activo en el pipeline CI <code>[[unique_id]]</code> .
Análisis Estático	Activo	Configurado y activo en el pipeline CI <code>[[unique_id]]</code> .
Deuda Técnica	Documentada	Problema de Room/KSP documentado en <code>TECH_DEBT.md</code> con solución temporal y criterios de resolución <code>[[unique_id]]</code> .

En conjunto, estos elementos —un pipeline robusto, una gestión de deuda transparente y una excelente cobertura de pruebas unitarias— posicionan a Bocatta POS en un estado de producción inminente. El sistema es estable, se mantiene en buen estado y está preparado para enfrentar los desafíos de un entorno de producción real.

Sistema de Diseño UI y Adaptabilidad Multigiro

El sistema de diseño de Bocatta POS, denominado "Meridian", ha sido concebido desde el principio como una base sólida y flexible que permita no solo una experiencia de usuario superior, sino también la adaptabilidad a diferentes giros comerciales. El sistema se basa en un conjunto de tokens de diseño semánticos y un conjunto de componentes reutilizables que guían la creación de una interfaz de usuario coherente, accesible y escalable. La implementación actual respeta estos principios, aunque la fase final para lograr un verdadero multi-giro todavía está pendiente.

Los **Tokens de Color Semánticos** son la columna vertebral del sistema de diseño. En lugar de usar colores duros como `Color.White` o `Color(0xFF1A1A1A)`, la aplicación utiliza exclusivamente `MaterialTheme.colorScheme.* [[unique_id]]`. Esto asegura que la interfaz de usuario respete el modo oscuro del sistema operativo sin necesidad de lógica extra. La paleta está diseñada para ser semántica, donde cada color tiene un propósito:

- **primary**: Verde de la marca (`#1B5E20`), usado para acciones primarias y precios.
- **error**: Rojo (`#C62828`), para estados negativos como el agotamiento de stock.
- **warning**: Naranja (`#EF6C00`), para advertencias como el stock bajo.
- **background y surface**: Blancos y grises claros para fondos y tarjetas.
- **onSurface**: Texto principal en negro o blanco, dependiendo del fondo.
- **outline**: Gris claro para bordes sutiles y divisores `[[unique_id]]`.

La **Tipografía y Jerarquía** están definidas para maximizar la legibilidad en un entorno operativo. El objetivo es que los precios y cantidades sean legibles a una distancia de dos metros, minimizando la necesidad de que el personal se acerque a la pantalla para confirmar una venta `[[unique_id]]`. Se establecen tamaños de fuente claros: 20sp para precios, 16sp para nombres de productos (truncados a dos líneas) y 12-14sp para metadatos como unidades o etiquetas de stock `[[unique_id]]`.

La **Geometría y Espaciado** siguen un sistema de diseño basado en un grid de 8dp, lo que crea una sensación de consistencia y pulcritud en toda la interfaz. Se definen radios de borde específicos para diferentes tipos de componentes (12dp para tarjetas, 8dp para campos de entrada) y se establece un tamaño mínimo de 48dp para todos los elementos táctiles, garantizando una experiencia de usuario cómoda y accesible `[[unique_id]]`. El diseño es adaptable, utilizando `BoxWithConstraints` para detectar el ancho de la pantalla y cambiar entre un layout vertical para móviles (con un panel de carrito que aparece como un `ModalBottomSheet`) y un layout horizontal para tabletas y terminales POS (con el catálogo y el carrito dispuestos en columnas fijas) `[[unique_id]]`.

Finalmente, el sistema se basa en un conjunto de **Componentes Reutilizables** definidos en `BocattaComponentsV2.kt`. Componentes como `ProductCardV2`, `StatusBadge` y `BocattaButton` encapsulan la lógica de renderizado y comportamiento, asegurando que se mantenga una apariencia y un funcionamiento consistentes en toda la aplicación. Por ejemplo, `ProductCardV2` siempre muestra un nombre, un precio y un badge de stock, independientemente de en qué pantalla se encuentre `[[unique_id]]`.

La siguiente tabla resume los parámetros de diseño clave:

Parámetro	Valor / Regla	Justificación
Paleta de Colores	Basada en <code>colorScheme</code> de Material3 (Primary, Error, Warning, Surface, etc.)	Modo oscuro nativo, accesibilidad WCAG AA, identidad de marca. <code>[[unique_id]]</code>
Tipografía	Inter/Ruboto Flex variable. Jerarquía definida (20sp, 16sp, 12sp)	Legibilidad a distancia, lectura rápida de información crítica. <code>[[unique_id]]</code>
Espaciado	Grid de 8dp. Radios de borde: 12dp (tarjetas), 8dp (inputs), 24dp (botones).	Coherencia visual, diseño responsive, áreas de toque accesibles. <code>[[unique_id]]</code>
Layouts	Adaptable con <code>BoxWithConstraints</code> . Breakpoint en 720dp.	Experiencia óptima en móvil, tableta y terminal POS. <code>[[unique_id]]</code>
Componentes	<code>BocattaComponentsV2.kt</code> (Tarjeta de producto, Badge de estado, Botón)	Reutilización, consistencia, mantenimiento simplificado. <code>[[unique_id]]</code>

Si bien el sistema de diseño está completamente definido y se está aplicando, la última pieza del rompecabezas para el multi-giro aún falta: una capa de presentación dinámica. Actualmente, la UI está codificada para mostrar ciertos campos. Para que el sistema sea verdaderamente agnóstico, se necesita un motor de formularios en la capa de presentación que pueda leer el giro y el mapa `attributes` de un `ProductV2` y generar automáticamente la interfaz de usuario (ej. campos de texto para una receta, un selector de duración para un servicio) `[[unique_id]]`. Una vez implementado, se podrá añadir un nuevo giro comercial simplemente modificando los datos en Firestore, sin escribir una sola línea de código de interfaz de usuario.

Hoja de Ruta Hacia la Madurez: Brechas Identificadas y Plan de Acción

A pesar de los avances significativos y de que Bocatta POS ha alcanzado un estado de madurez excepcional (calificación de 97/100), persisten algunas brechas técnicas que deben ser cerradas para llegar a un nivel de perfección. Estas brechas se centran

principalmente en la limpieza del código, el aumento de la cobertura de pruebas y la culminación del objetivo de agnosticismo de giro. A continuación se detalla el estado de estas brechas y se propone un plan de acción secuencial para abordarlas.

La primera brecha es la **convivencia de código legacy**. Aunque se han creado mapeadores (**ProductMappers**) para aislar la transición, todavía existe un camino dual entre el modelo de datos antiguo (**ProductV2**) y el nuevo modelo de dominio (**ProductV2**). Esto aumenta la superficie de error y la complejidad del sistema. La acción correcta es eliminar gradualmente el código legacy una vez que todas las funcionalidades críticas hayan sido portadas completamente al nuevo modelo. Esto implica actualizar la capa de datos para que solo trabaje con el modelo de dominio y, finalmente, eliminar los modelos y mapeadores antiguos.

La segunda brecha es la **cobertura de pruebas**. El pipeline CI actual se centra en las pruebas unitarias del dominio, lo cual es excelente, pero le falta una capa crítica: las pruebas de integración y de la interfaz de usuario. Faltan pruebas que validen el comportamiento visual y los flujos de usuario completos en dispositivos reales. Es imperativo implementar pruebas de la interfaz de usuario con **ComposeTestRule** para validar la responsividad, los estados de carga y la navegación, y utilizar el emulador de Firestore para realizar pruebas de integración que simulen la interacción con la base de datos bajo diferentes condiciones de red `[[unique_id]]`.

La tercera y más importante brecha es la **ausencia de una UI dinámica**. Como se mencionó, el sistema es conceptualmente agnóstico al giro gracias al modelo de datos (**attributes**), pero la capa de presentación aún no aprovecha esta flexibilidad. La interfaz de usuario actual es estática y necesita ser reemplazada por un motor de formularios dinámicos. Este motor sería responsable de interpretar los esquemas de producto y generar los layouts y campos de formulario correspondientes. Esta es la tarea final para lograr un verdadero multi-giro, donde los nuevos negocios puedan ser configurados y operados sin necesidad de desarrollar software nuevo.

A continuación se presenta un plan de acción secuencial para cerrar estas brechas y alcanzar la madurez deseada:

Fase	Acción Clave	Prioridad	Objetivo
Fase 1	Validación en Campo	Alta	Ejecutar pruebas de estrés (ventas offline, reconexión, duplicados) para confirmar la fiabilidad de la cola offline de SQLite [[unique_id]].
Fase 2	Limpieza de Código Legacy	Media	Eliminar todos los puntos de contacto con el código y los modelos antiguos, consolidando en el nuevo modelo de dominio.
Fase 3	Desarrollo de la UI Dinámica	Alta	Crear un motor de formularios en la capa de presentación que renderice campos dinámicamente basados en el giro y atributos de los productos.
Fase 4	Aumento de Cobertura de Pruebas	Alta	Implementar pruebas de la interfaz de usuario con Compose Test y pruebas de integración con el emulador de Firestore.
Fase 5	Migración a KSP/Room	Baja	Una vez que la comunidad libere versiones compatibles, planificar la migración de la persistencia offline de SQLite nativo a Room para recuperar la validación en tiempo de compilación.

Siguiendo este plan, Bocatta POS estará listo para una transición completa a un ERP modular, robusto y listo para la producción. El sistema no solo habrá superado los obstáculos técnicos iniciales, sino que habrá evolucionado a una plataforma de clase empresarial, preparada para escalar y servir a una amplia gama de clientes comerciales.